

Video Processing 0512-4263

Two exams, fully solved (Moed A & Moed B, 2022)

Solutions grounded only in the course HTML book

How each question maps to the book

The exam has 8 fixed “slots”. Each maps to one chapter:

- **Optic Flow** → Ch.2 (Horn–Schunck)
- **Tracking** → Ch.4 (Kalman / particle filter)
- **Intrinsic images / Dynamic texture** → Ch.12 (Weiss median) / Ch.5 (state-space texture)
- **Clone detection / PatchMatch** → Ch.11
- **Geodesic distance** → Ch.9 (colorization) & Ch.10 (matting)
- **Segmentation** → Ch.9 geodesic label propagation
- **Clustering / Bidirectional similarity** → Ch.6 (k-means) / Ch.11 (completeness–coherence)

Two questions are not covered by Book 2 — Moed A Q4 (Evolving Time Fronts / dynamosaicing) and Moed B Q4 (Light Fields). They were “papers discussed in class” this book does not reproduce. They are marked [not in Book 2] and answered only with reasoning the book does support, with the gap called out honestly.

1 Exam 1 — Moed A (19/6/2022)

1.1. OPTIC FLOW

[maps to: Ch.2]

13 pts

Question. You have a semantic-segmentation network: image $I \rightarrow$ label map S (each pixel a class $0..c-1$; all cars share one label). Modify Horn–Schunck to take I_1, I_2 and label maps S_1, S_2 and output the optic flow.

How I thought about it

Base Horn–Schunck minimises

$$\min_{u,v} \iint (I_x u + I_y v + I_t)^2 + \alpha (|\nabla u|^2 + |\nabla v|^2) dx dy.$$

Its known weakness (Ch.2): the smoothness term smooths flow *everywhere*, blurring motion across object boundaries. The label map S is exactly a boundary detector: where the label changes there is a real boundary where the flow may jump. So: keep the data term, but stop smoothness from diffusing flow across label boundaries.

Solution

Introduce a per-edge weight, on inside a class and off across classes. For neighbours p, q :

$$w(p, q) = \begin{cases} 1 & S_1(p) = S_1(q) \\ \varepsilon \approx 0 & S_1(p) \neq S_1(q) \end{cases}$$

and minimise the segmentation-aware energy

$$\min_{u,v} \iint (I_x u + I_y v + I_t)^2 + \alpha w(x) (|\nabla u|^2 + |\nabla v|^2) dx dy.$$

The Euler–Lagrange derivation of Ch.2 then replaces the ordinary Laplacian by a *weighted graph Laplacian*: each neighbour contributes $-w(p, q)$ instead of -1 , so edges crossing a label boundary have weight 0 and flow no longer leaks across it — sharp motion discontinuities appear exactly at object silhouettes. S_2 gives an optional label-consistency data term $\lambda [S_2(x + u, y + v) \neq S_1(x)]$ (penalise sending a “car” pixel onto “road”); it is soft because instances of a class share the label.

Answer. Horn–Schunck with a label-gated smoothness weight ($w = 0$ across label changes) $\Rightarrow$ boundary-preserving dense flow; optionally a label-consistency data term using S_2 .

1.2. TRACKING — loaded die

[maps to: Ch.4 Condensation]

12 pts

Question. Simulate rolling a loaded six-sided die with probabilities $(p_1, \dots, p_6)$ using only $\text{rand}() \sim U[0, 1]$.

How I thought about it

This is exactly the “select” step of the Condensation / particle filter (Ch.4), which samples an index in proportion to weights using one uniform draw. Faces are particles; p_i are the weights.

Solution

Build the CDF $C_0 = 0$, $C_i = C_{i-1} + p_i$ ($i = 1..6$), $C_6 = 1$. Then: draw $r = \text{rand}()$; return the smallest j with $C_j > r$ (i.e. $C_{j-1} \leq r < C_j$). Face j is returned when r falls in $[C_{j-1}, C_j)$, an interval of length p_j ; since r is uniform, $P(\text{face } j) = p_j$. Identical to the Ch.4 select step “generate r ; find smallest j with $C^{(j)} > r$ ”.

Answer. Inverse-CDF sampling — cumulative sums C_i , one uniform r , pick the bin r falls into. Same as particle-filter resampling.

1.3. INTRINSIC IMAGES — online reflectance

[maps to: Ch.12 Weiss]

13 pts

Question. Give an *online* algorithm for the reflectance image R_t from a stream $I_1, \dots, I_t$. Store at most 256 images (or equivalent). Hint: intensities are in $[0, 255]$.

How I thought about it

Ch.12 (Weiss): with $I(x, t) = R(x)L(x, t)$, in $\log i = r + l$. Filter each frame with derivative filters f_n ; assuming filtered illumination is sparse, the ML filtered-reflectance is the per-pixel temporal median $\hat{r}_n(x) = \text{median}_t(f_n * i)(x, t)$, and r is reconstructed by the fixed pseudo-inverse (Ch.12 eq. 6), $R = e^r$. Only the median blocks an online version. Hint “[0, 255], store 256 images” $\Rightarrow$ a running median from a **256-bin histogram**.

Solution

Keep, per pixel and per filter, a 256-bin histogram over the (quantised) response range — 256 numbers per pixel, independent of t . For each new frame I_t :

1. Compute $i = \log I_t$ and responses $f_n * i$.
2. Increment each pixel’s histogram bin ($O(1)$).
3. Running median $\hat{r}_n(x) =$ smallest bin whose cumulative count reaches $t/2$.
4. Reconstruct R_t from the current medians with the fixed deconvolution filter g (Ch.12 eq. 6–7, precomputed once, depends only on the filters).

Memory = $256 \times (\# \text{filters})$ per pixel (Weiss uses only horizontal + vertical derivatives $\Rightarrow$ two histograms); work $O(1)/\text{pixel}/\text{frame}$; emit R_t every frame.

Answer. Per-pixel 256-bin histogram of the filtered log-frames $\rightarrow$ running temporal median $\rightarrow$ Weiss reconstruction. Constant memory, truly online.

1.4. EVOLVING TIME FRONTS — Iguazu

[not in Book 2]

12 pts

Question. A video pans left→right over the Iguazu falls. Synthesize a video that pans right→left while the water still falls downward.

Honesty note. Book 2 has no chapter on this (the “dynamosaicing / evolving time fronts” paper). The answer uses only space-time reasoning the book supports; the named technique is outside Book 2.

How I thought about it

Two motions are entangled: the global horizontal *camera pan* and the local downward *water* (a dynamic texture, cf. Ch.5). Plain reverse playback reverses both, so water would flow up. They must be decoupled: reverse the spatial scan while keeping local time forward.

Solution (space-time volume)

Stack all frames into an xyt volume. An ordinary output frame is a planar slice at $t = \text{const}$ (a “time front”). To pan right→left with water still falling, sweep a *slanted / evolving time-front surface* through the volume that moves across x in the *opposite* direction to the original scan, but where the local time coordinate along the sweep still *increases*. Each output frame is a cut of the volume with this surface; advancing the surface in $-x$ produces the reversed pan, while local t moving forward keeps the water falling down.

Answer. Build the xyt volume; synthesize each frame as a non-planar time-front cut; sweep the front in reverse- x while local time advances forward. (*Technique itself outside Book 2.*)

1.5. CLONE DETECTION

[maps to: Ch.11 PatchMatch]

13 pts

Question. An object was removed by inpainting/cloning. Given only the doctored image, automatically detect the cloned regions.

How I thought about it

Cloning = two regions of the *same* image are near-identical copies. PatchMatch (Ch.11) gives each patch its nearest-neighbour patch elsewhere, and a coherent region produces a *constant offset* in the NN field. A copy is a region whose patches all match another location with near-zero distance and one shared offset. So run PatchMatch of the image against itself.

Solution

1. PatchMatch with source = target = input image, giving offset field $f(x)$ and distance $d(x) = \|P_x - Q_{x+f(x)}\|^2$; *forbid the trivial match* (offsets within a few patch-widths of x).
2. Clone candidates: pixels with $d(x) \approx 0$.
3. Among candidates keep large connected components sharing one dominant offset v (histogram the offsets); a genuine copy-move appears as a matched pair, region A with $+v$ and region B with $-v$.
4. Output those components as the clone mask.

Answer. Self-PatchMatch (image vs itself, trivial match excluded) → flag regions with near-zero patch distance *and* a shared coherent offset → those matched components are the clones.

1.6. GEODESIC DISTANCE — the γ parameter

[maps to: Ch.9/10]

12 pts

Question. With $d(a, b) = \inf_{\Gamma} \int_0^{l(\Gamma)} \sqrt{1 + \gamma^2 (\nabla I(s) \cdot \Gamma'(s))^2} ds$, decide for maps (c) and (d) whether $\gamma = 0$ or $\gamma > 0$ (darker = shorter). Explain.

How I thought about it

γ sets how much the *image* influences distance. Read the integrand:

- $\gamma = 0$: integrand $= \sqrt{1} = 1$, so $d = \int ds = \text{arc length}$ — purely spatial/Euclidean from the object region Ω , completely image-blind. Smooth halo; background swirls leave no trace.
- $\gamma > 0$: the term $\gamma^2(\nabla I \cdot \Gamma')^2$ adds cost wherever a path runs across an image gradient. Distance is image-aware: it grows fast crossing edges, small inside homogeneous regions, so image structure (swirls, silhouette) becomes *visible* and the map “sticks” at edges.

Determination

Cue: “does the map show the image’s own structure?”

- (c) — smooth halo, background swirls invisible $\Rightarrow \gamma = 0$ (geometry only).
- (d) — swirl textures / edges show up in the field $\Rightarrow \gamma > 0$ (crossing them costs distance).

Answer. (c) uses $\gamma = 0$ (image-blind Euclidean halo); (d) uses $\gamma > 0$ (edges visible). Rule: map ignores texture $\Rightarrow \gamma = 0$; texture prints through $\Rightarrow \gamma > 0$.

1.7. SEGMENTATION — 3D point cloud + scribbles [maps to: Ch.9] 13 pts

Question. Given an unordered 3D point cloud and coloured user scribbles, reproduce the segmentation.

How I thought about it

Colorization/matting (Ch.9/10): propagate seed labels by geodesic distance. The domain is a point cloud, not a pixel grid — only the graph construction is new; then the same Dijkstra machinery applies.

Solution

1. Build a graph $G = (V, E)$: nodes = points; connect each to its k nearest neighbours in 3D (the list is unordered $\Rightarrow$ use spatial proximity).
2. Edge weights so distance grows across boundaries: $w(p, q) = \|p - q\|$, optionally plus a surface-normal / feature difference so a sharp geometric boundary is expensive.
3. Seeds Ω_k = points carrying scribble colour k .
4. Per label k , compute geodesic distance $D_k(v) = d_L(v, \Omega_k)$ with Dijkstra (Ch.9).
5. Assign $\text{label}(v) = \arg \min_k D_k(v)$; boundary = where two D_k tie.

Answer. kNN graph on the points $\rightarrow$ edge weights from 3D/normal dissimilarity $\rightarrow$ per-scribble geodesic distance (Dijkstra) $\rightarrow$ label each point by its nearest scribble. Ch.9 moved from a pixel grid to a point graph.

1.8. BIDIRECTIONAL SIMILARITY — is D symmetric? [maps to: Ch.11] 12 pts

Question. $D(P, Q) = \max_{p \in P} \min_{q \in Q} d(p, q)$. Is $D(P, Q) = D(Q, P)$? Prove or give a counter-example.

How I thought about it

This is the directed (one-sided) distance — exactly D_{com} vs D_{coh} in Ch.11: the two directions differ, which is why retargeting needs both. Expect “not symmetric”; find a two-point counter-example.

Solution — counter-example

On the real line, $P = \{0\}$, $Q = \{0, 10\}$:

$$D(P, Q) = \min\{d(0, 0), d(0, 10)\} = 0, \quad D(Q, P) = \max\{d(0, 0), d(10, 0)\} = \max\{0, 10\} = 10.$$

Since $0 \neq 10$, D is **not symmetric**.

Answer. No. $P = \{0\}, Q = \{0, 10\}$ gives $D(P, Q) = 0$ but $D(Q, P) = 10$. This is the completeness-vs-coherence asymmetry, symmetrised in retargeting as $\alpha D_{\text{com}} + (1 - \alpha) D_{\text{coh}}$.

2 Exam 2 — Moed B (1/8/2022)

2.1. OPTIC FLOW — sparse LiDAR depth

[maps to: Ch.2]

13 pts

Question. A car has a grayscale camera and a LiDAR giving depth at a *sparse* pixel set. Modify Horn–Schunck to use I_1, I_2 and sparse depth maps D_1, D_2 and output the flow.

How I thought about it

Same weakness as before (HS over-smooths across boundaries). Depth gives two signals: (1) depth discontinuities are object boundaries $\Rightarrow$ smoothness off there; (2) where depth exists it is a hard measurement anchoring the flow. Sparse $\Rightarrow$ both act only at pixels with depth.

Solution

Depth-aware smoothness. For neighbours p, q with depth, set $w(p, q) = \exp(- (D_1(p) - D_1(q))^2 / 2\sigma_d^2)$ (near 1 within a surface, near 0 across a depth edge); where depth is missing, fall back to constant α . Minimise

$$\iint (I_x u + I_y v + I_t)^2 + \alpha w(x) (|\nabla u|^2 + |\nabla v|^2) dx dy,$$

a weighted-Laplacian solve preserving motion boundaries at depth edges. *Sparse depth-constancy data term.* Only at LiDAR pixels, a tracked pixel carries its depth to the match, so add $\lambda \sum_{x \in \text{LiDAR}} (D_2(x + u, y + v) - D_1(x))^2$, pinning the flow at the measured points and letting smoothness interpolate to the rest.

Answer. HS with (i) smoothness weight small across depth discontinuities and (ii) a sparse depth-constancy data term at LiDAR pixels. Sparse anchors + boundary-aware smoothness.

2.2. TRACKING — the Bayes recursion

[maps to: Ch.4]

12 pts

Question. State the Markov and conditional-independence assumptions and show $p(x_{1:t} | z_{1:t}) \propto p(z_t | x_t) p(x_t | x_{t-1}) p(x_{1:t-1} | z_{1:t-1})$.

Assumptions

Markov (motion): $p(x_t | x_{1:t-1}) = p(x_t | x_{t-1}) \Rightarrow p(x_{1:t}) = p(x_t | x_{t-1}) p(x_{1:t-1})$.

Conditional independence of observations: $p(z_{1:t} | x_{1:t}) = p(z_t | x_t) p(z_{1:t-1} | x_{1:t-1}) = \prod_{\tau=1}^t p(z_\tau | x_\tau)$.

Derivation

$$\begin{aligned} p(x_{1:t} | z_{1:t}) &= \frac{p(z_{1:t} | x_{1:t}) p(x_{1:t})}{p(z_{1:t})} \propto p(z_{1:t} | x_{1:t}) p(x_{1:t}) \\ &= \underbrace{p(z_t | x_t) p(z_{1:t-1} | x_{1:t-1})}_{\text{cond. indep.}} \cdot \underbrace{p(x_t | x_{t-1}) p(x_{1:t-1})}_{\text{Markov}} \\ &= p(z_t | x_t) p(x_t | x_{t-1}) \underbrace{[p(z_{1:t-1} | x_{1:t-1}) p(x_{1:t-1})]}_{\propto p(x_{1:t-1} | z_{1:t-1})} \\ &\propto p(z_t | x_t) p(x_t | x_{t-1}) p(x_{1:t-1} | z_{1:t-1}). \end{aligned}$$

The three factors are the **likelihood**, the **motion model**, and the **recursion** — the basis of the particle filter. ■

2.3. DYNAMIC TEXTURE — Word2Vec text synthesis [\[maps to: Ch.5\]](#) 13 pts

Question. Word2Vec maps words to vectors. Use it to synthesize text: short input $\rightarrow$ a page of output.

How I thought about it

The slot is “dynamic texture” $\rightarrow$ Ch.5: a texture is a low-dimensional linear state-space process driven by noise, learned from a sample then run forward. In Ch.5 a frame becomes a vector via SVD; here a word already becomes a vector via Word2Vec. So the word sequence is a trajectory in embedding space; synthesis = drive it forward with noise, then decode vectors back to words.

Solution

1. **Vectorise:** $y_t = \text{W2V}(\text{word}_t)$ for the input — analogue of “frame $\rightarrow$ vector”.
2. **System ID:** model $x_{t+1} = Ax_t + Bv$, $y_t = Cx_t + w$. Take $x_t = y_t$ (embeddings are already Euclidean; optionally reduce with SVD) and fit $\hat{A} = \arg \min_A \sum_t \|x_{t+1} - Ax_t\|^2$ (or ARMA $x_t = \sum_i p_i x_{t-i} + \varepsilon$).
3. **Generate:** seed with the input’s last state(s); iterate $x_{t+1} = \hat{A}x_t + Bv$ for a page. Noise v is essential (Ch.5) — without it the trajectory is deterministic and dies/repeats.
4. **Decode:** word = $\arg \min_w \|x - \text{W2V}(w)\|$ (nearest neighbour in the vocabulary).

Like all dynamic-texture synthesis, this reproduces local statistics/style, not deep grammar.

Answer. Treat the Word2Vec-vector sequence as a dynamic texture: fit an ARMA/linear state-space model (Ch.5), run it forward with noise, decode each generated vector to its nearest word.

2.4. LIGHT FIELDS — novel view from a rotating camera [\[not in Book 2\]](#) 12 pts

Question. A camera on a rotating arm records a video around an axis. Synthesize a novel view from a virtual camera *inside* the circle.

Honesty note. Book 2 has no light-fields chapter. The answer uses only the multi-view geometry the book supports (Ch.1 projection/homography) plus interpolation; the named “light field / concentric mosaic” method is outside Book 2.

How I thought about it

The rotating camera samples a dense family of scene rays. A novel view is a new bundle of rays through the virtual centre. If for each virtual ray I find the recorded ray that coincides with it, I copy that pixel — ray re-indexing + interpolation, built on the Ch.1 projection matrix.

Solution

1. **Calibrate:** know each recorded frame’s projection matrix P_k (Ch.1: intrinsics + known arm angle/position give R_k, t_k); give the virtual camera P_v .
2. **Per virtual pixel = a ray:** back-project via P_v to a 3D ray.
3. **Match:** among all frames, find the camera/pixel whose optical ray best coincides with that virtual ray; sample its colour.
4. **Interpolate:** when a virtual ray falls between two recorded rays, blend the nearest samples by angular closeness.

The ring of views densely covers ray-space, so nearby recorded rays approximate any virtual ray well (the light-field / IBR idea).

Answer. Reconstruct each virtual pixel from the recorded camera ray that coincides with the virtual ray (via the Ch.1 projection matrices), interpolating between nearest recorded views. (*Light fields outside Book 2.*)

2.5. PATCHMATCH — extend to k-NN

[maps to: Ch.11]

13 pts

Question. 1-NN PatchMatch returns the single best match per patch. Extend it to k-NN: the top- k nearest neighbours per patch.

How I thought about it

PatchMatch is “keep the best offset; improve by propagation + random search”. For k-NN keep the best k offsets and let both operations draw candidates from neighbours’ whole k -lists. The per-patch structure becomes a bounded heap of size k .

Solution

1. **Init:** k random distinct offsets per patch (with distances), in a size- k max-heap ordered by patch distance (top = current worst).
2. **Propagation** (alternating raster scans): candidate pool = x ’s k offsets $\cup$ left neighbour’s $k \cup$ above neighbour’s k ; keep the k smallest-distance *distinct* offsets.
3. **Random search:** around each held offset sample with shrinking radius; if a sample beats the worst and is not a near-duplicate, insert and evict the worst.
4. Iterate 4–6 passes; output each patch’s k -offset list.

Distinctness is the subtlety: reject candidates too close to already-held offsets, else the heap fills with k copies of one match. Cost $\approx k \times$ the 1-NN cost, still far below brute force.

Answer. Replace the single best offset with a size- k heap; propagation and random search draw from neighbours’ full k -lists and keep the k best *distinct* offsets.

2.6. GEODESIC DISTANCE — path along Lena’s hat edge

Dijkstra]

[maps to: Ch.9

12 pts

Question. The user clicks A and B ; the system auto-draws the path between them that follows the edge of the hat. Compute that path.

How I thought about it

“Follows the edge” = shortest path in a weighted grid (Ch.9). But note the *sign flip*: colorization made crossing an edge expensive ($w = |\nabla Y|$) so paths avoid edges. Here we want the path to ride the edge, so it must be cheap where the gradient is strong — weight inversely related to gradient magnitude.

Solution

1. Grid graph; edge weight small on strong image edges, e.g. $w = \frac{1}{|\nabla I| + \epsilon}$ or $w = (\max |\nabla I| - |\nabla I|)$. Walking along a high-gradient contour costs almost nothing.
2. Dijkstra from A ; read the shortest path to B (Ch.9 Bellman/DP). It hugs the hat’s rim.
3. Draw it (“intelligent scissors” — same algorithm as colorization, edge-*following* weights).

Answer. Grid graph with weights small where $|\nabla I|$ is large (edge-following), then Dijkstra shortest path $A \rightarrow B$. Ch.9 machinery, inverted edge cost.

2.7. SEGMENTATION — social network interest mix [maps to: Ch.9 blending]

13 pts

Question. A graph $G = (V, E)$ with tie strengths $w(u, v)$; a few users labelled (finance expert u_1 , fashion expert u_2). Learn every node’s *mix* of interests (e.g. 30% finance / 70% fashion).

How I thought about it

Labelled experts = scribbles; the wanted output is a *soft mixture* summing to 1 = the **blending** formula of colorization (Ch.9), not a hard label. The graph is given; only convert strength to a distance and reuse geodesic distance + soft blend.

Solution

1. **Strength** $\rightarrow$ **distance**: a strong tie means “close”, so edge length $\ell(u, v) = 1/w(u, v)$ (or $-\log w$); geodesic distance is short along strong-connection chains.
2. **Seeds** Ω_k = users labelled with interest k .
3. **Distance maps** $D_k(v)$ = geodesic distance to seed set k (Dijkstra on G).
4. **Soft mixture** with $W(D) = 1/D^b$:

$$\text{interest}_k(v) = \frac{W(D_k(v))}{\sum_j W(D_j(v))}.$$

These are non-negative and sum to 1 — the percentage split. Literally colorization’s $c(x) = \frac{\sum_k W(D_k)c_k}{\sum_k W(D_k)}$ with categories in place of colours.

Answer. Turn tie strength into edge length $1/w$, compute geodesic distance from each labelled expert, output the distance-weighted normalized blend $W(D_k)/\sum_j W(D_j)$ as each node’s interest percentages.

2.8. CLUSTERING — does k-means terminate?

[maps to: Ch.6]

12 pts

Question. Is k-means guaranteed to terminate? Explain.

Argument

k-means minimises the distortion $J = \sum_i \|x_i - c_{\text{enc}(i)}\|^2$ by alternating two steps. **Both steps are non-increasing in J :**

- *Assignment* (each $x_i \rightarrow$ nearest centre): by definition of “nearest”, each term can only stay or drop $\Rightarrow J$ does not increase.
- *Update* (centre $\leftarrow$ mean of members): the mean minimises $\sum_{i \in S_j} \|x_i - c\|^2$ (derivative 0 gives $c_j = \frac{1}{|S_j|} \sum_{i \in S_j} x_i$, Ch.6) $\Rightarrow J$ does not increase.

So J is monotonically non-increasing. **Finiteness:** there are $\leq k^n$ assignments; each fixes its centres (means) and hence a specific J . Since J never increases, once the algorithm leaves an assignment (only when the assignment strictly lowers J , using a fixed tie-break) it can never return — that would raise J . A monotone sequence over finitely many values cannot decrease forever, so the assignment stops changing after finitely many steps and the algorithm halts.

Answer. Yes. Distortion decreases (never increases) at every step and there are finitely many assignments, so no assignment repeats and it must stop in finite time. Caveat: converges to a *local* minimum (global k-means is NP-hard) and depends on initialization — but termination is guaranteed.

3 The question patterns

Both exams reuse one skeleton — one question per topic, 12–13 points each. Recognise the slot and you know the chapter:

#	Slot	Ch.	What it always asks
1	Optic Flow	2	“Modify Horn–Schunck to use extra input X ” (labels, depth, ...)
2	Tracking	4	Particle-filter resampling, or the Bayes recursion derivation
3	Intrinsic / Dyn. texture	12/5	Weiss median (online variant) or state-space synthesis in a new domain
4	“Paper” slot	—	A paper from class (time fronts, light fields) — often outside the notes
5	PatchMatch / retargeting	11	Manipulate the NN field: clone detection, k-NN, inpaint
6	Geodesic distance	9/10	Distance map / shortest path; edge-avoiding vs following; γ
7	Segmentation	9	Propagate seed labels by geodesic distance on a graph
8	Clustering / theory	6/11	“Is it guaranteed / symmetric?” — monotone proof or counter-example

Four meta-recipes that solve almost all of them

- **“Modify algorithm A to accept new input B.”** Find where B signals a boundary or a reliable measurement; inject it as a smoothness weight (turn smoothing off across B ’s boundaries) and/or a data term (anchor the solution where B is measured). — Optic-flow slot, every time.
- **“Design an online / efficient version.”** Replace “store everything” with a running statistic: a per-pixel histogram (running median/mode), a low-rank factorization, or PatchMatch instead of brute-force NN.
- **“Adapt an image method to a new domain”** (3D points, a graph, words): build the right graph or vector space, then reuse the same machinery — geodesic distance (Dijkstra), ARMA state-space, or the NN field.
- **“Is X guaranteed / symmetric / a metric?”** A monotone-decrease + finiteness argument, or a tiny two/three-point counter-example.

The unifying hammer: *define a cost, add a boundary-aware regularizer, minimise it iteratively* (least squares / Dijkstra-DP / EM-style alternation).

4 New practice questions (same patterns)

Ten fresh questions on the templates above, each with a short answer key.

4.1. Optic Flow — confidence map

[maps to: slot 1]

Given I_1, I_2 and a per-pixel confidence map $U(x) \in [0, 1]$ (high where textured/reliable). Modify Horn–Schunck.

Answer. Weight the *data* term by U : $\iint U(x)(I_x u + I_y v + I_t)^2 + \alpha(|\nabla u|^2 + |\nabla v|^2)$. Trust brightness only where U is high; in low- U regions the data term nearly vanishes and smoothness fills flow from confident neighbours.

4.2. Optic Flow — external edge map

[maps to: slot 1]

Given I_1, I_2 and a binary motion-boundary map B , modify Horn–Schunck.

Answer. Set smoothness weight $w(p, q) = 0$ for neighbour pairs separated by B (else 1) $\Rightarrow$ weighted-Laplacian solve that lets flow jump on B . Same structure as Moed A/B Q1.

4.3. Tracking — sample from a CDF

[maps to: slot 2]

Using only `rand()`, draw from a continuous distribution with known CDF F .

Answer. Inverse-CDF sampling: $r = \text{rand}()$, return $F^{-1}(r)$. The loaded die (Moed A Q2) is the step-function special case.

4.4. Tracking — filtering recursion

[maps to: slot 2]

Derive $p(x_t | z_{1:t}) \propto p(z_t | x_t) \int p(x_t | x_{t-1})p(x_{t-1} | z_{1:t-1}) dx_{t-1}$.

Answer. From Moed B Q2’s posterior recursion, marginalise $x_{1:t-1}$: the integral of $p(x_t | x_{t-1})p(x_{t-1} | z_{1:t-1})$ is the predicted prior $p(x_t | z_{1:t-1})$; multiply by the likelihood and normalise (Ch.4 “filtering”).

4.5. Intrinsic — online temporal mode

[maps to: slot 3]

Online, bounded-memory per-pixel temporal *mode* of a $[0, 255]$ video (background modelling).

Answer. Per-pixel 256-bin histogram; increment each frame ($O(1)$); mode = $\arg \max$ bin. Same trick as the online-median reflectance (Moed A Q3), read the peak instead of the median.

4.6. Dynamic texture — melody synthesis

[maps to: slot 3]

With a note-embedding (“music2vec”), synthesize a long melody from a short one.

Answer. Moed B Q3 with notes: embed the note sequence, fit an ARMA/linear state-space model (Ch.5), drive forward with noise, decode each vector to its nearest note.

4.7. PatchMatch — match across many images

[maps to: slot 5]

Extend PatchMatch to find, for each source patch, its NN across a set of M target images.

Answer. Let the offset carry an image index: the match is (m, y) . Propagation borrows neighbours’ (m, y) ; random search perturbs y and occasionally re-rolls m ; keep the best across all M (Ch.11 multi-source reshuffling).

4.8. PatchMatch — duplicated video segment

[maps to: slot 5]

Detect a looped/duplicated temporal segment inside one video.

Answer. 3D (space-time) self-PatchMatch, excluding the trivial temporal neighbourhood; a duplicate appears as voxels with near-zero distance and a constant *temporal* offset. Clone detection (Moed A Q5) in the time dimension.

4.9. Geodesic — stay in the middle of the road

[maps to: slot 6]

User clicks two points; draw the path that stays in the middle of a road (avoiding its edges).

Answer. Opposite weighting to the Lena-hat question (Moed B Q6): make edges expensive, $w = |\nabla I|$ (colorization weight), then Dijkstra $A \rightarrow B$. The path avoids high-gradient boundaries and runs through the smooth interior.

4.10. Clustering — is k-means monotone? can it cycle?

[maps to: slot 8]

Can the distortion ever increase? Can k-means cycle forever? What about a point equidistant from two centres?

Answer. Distortion never increases (both steps non-increasing — Moed B Q8). No cycle: finitely many assignments + monotone decrease $\Rightarrow$ no repeat. Ties need a fixed tie-break (e.g. lowest index) so each changed assignment strictly lowers J .

Study tip

For a new question, first name its **slot** (1–8), which fixes the chapter, then ask: “what is the extra input, and is it a *boundary signal* ($\rightarrow$ smoothness weight) or a *measurement* ($\rightarrow$ data term / seed)?” That answers slots 1, 6, 7 outright; the online/efficiency and domain-transfer recipes cover the rest.